

Transmission Control Protocol (*TCP*)

- ◆ Stream service
 - ◆ *Reliable, connection-oriented* service
 - ◆ In-order delivery to application process
- ◆ Flow control
- ◆ Congestion control
- ◆ Has significant *overhead*
 - ◆ Suitable for applications that need this degree of reliability

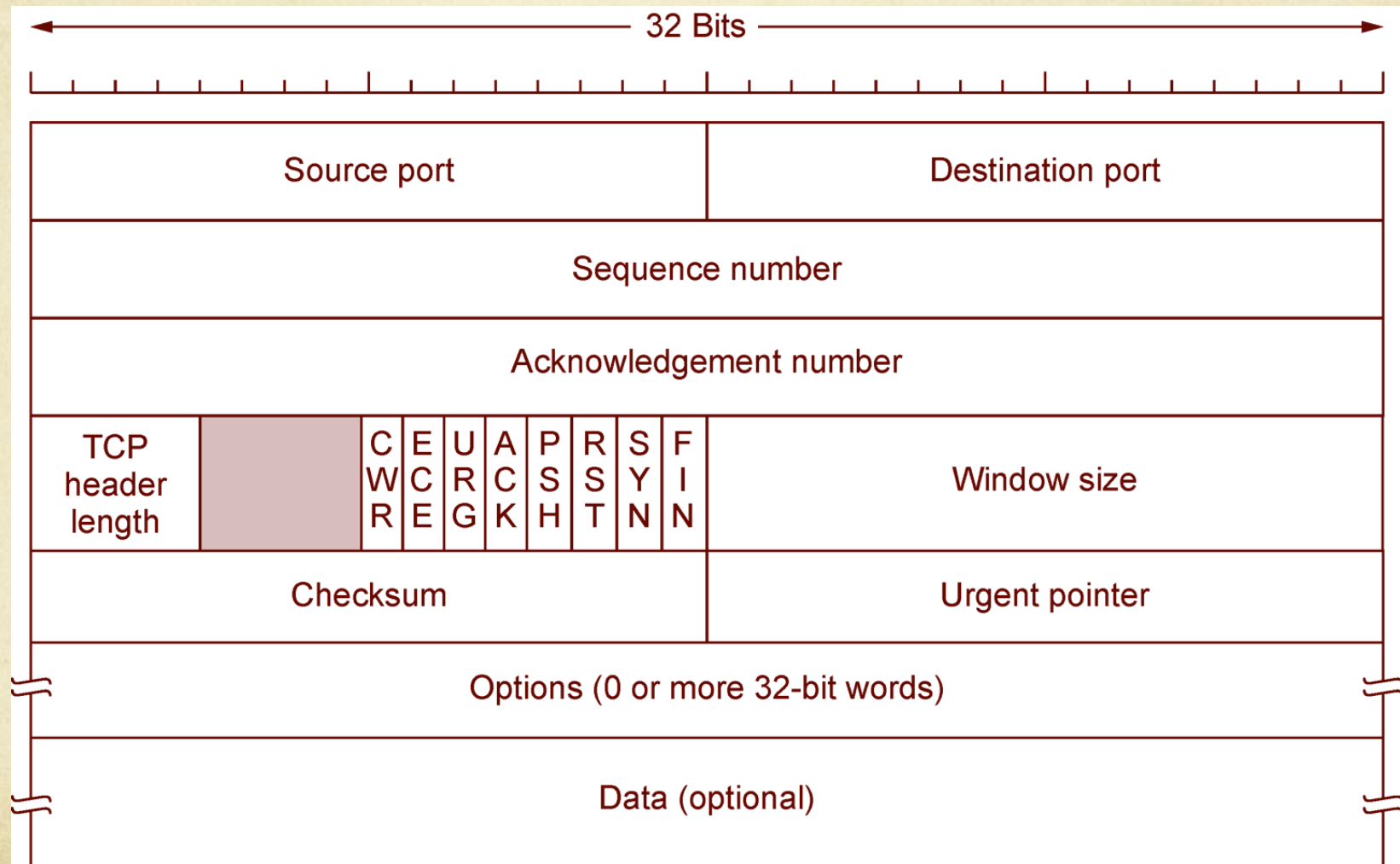
Connections in TCP

- ◆ Host being connected to must be ready to accept connections
 - ◆ *Listen* or *passive open* state
- ◆ Host that wants to establish new connection
 - ◆ *Connect* or *active open* state
- ◆ E.g., *server* is in passive open state & *clients* can establish connections with server

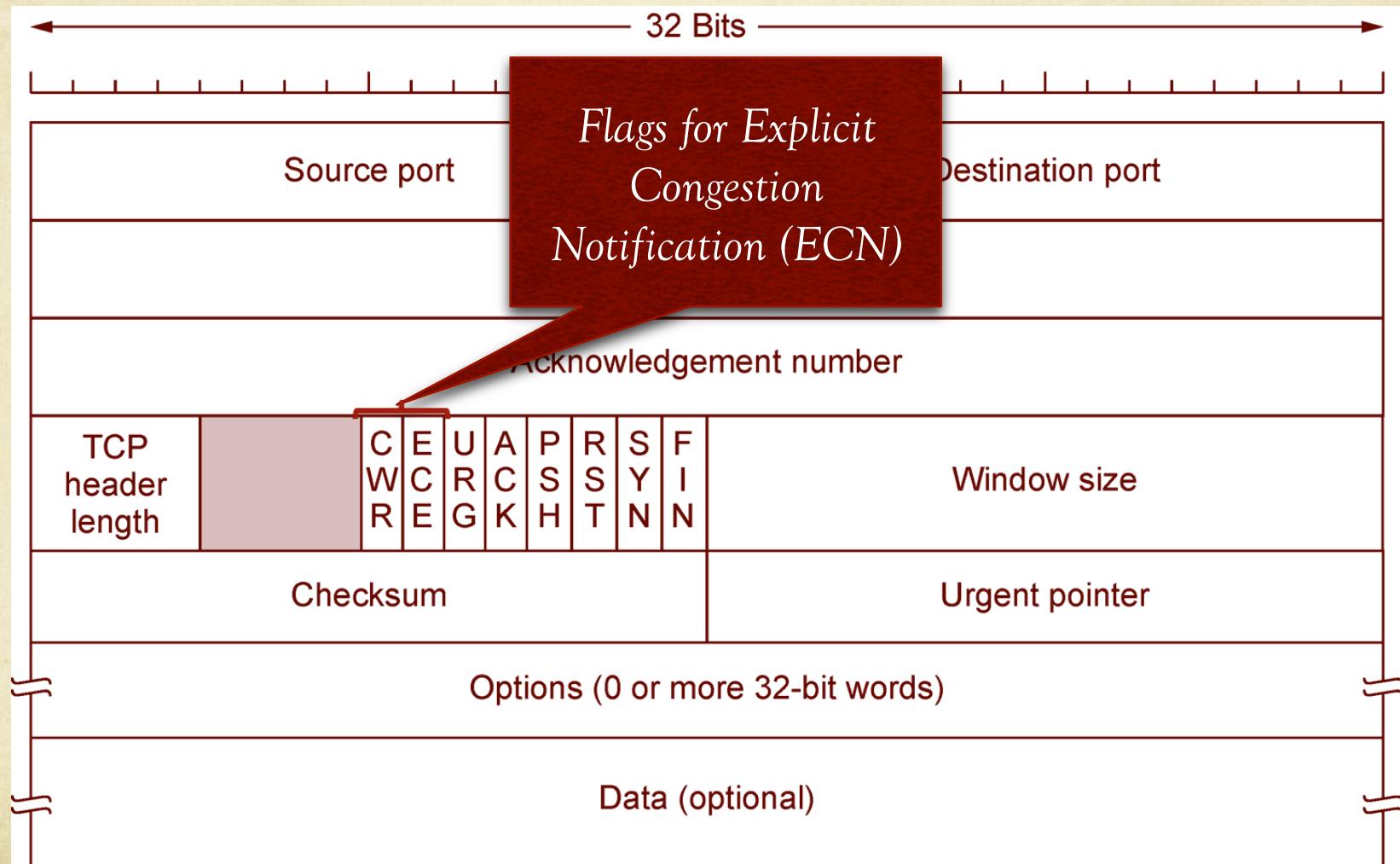
Reliable, in-order delivery

- ◆ Acknowledgement
 - ◆ Receiver acknowledges receipt of every segment (*ACK*)
- ◆ Retransmission
 - ◆ When segment sent, *retransmission timer* started
 - ◆ If *ACK* received before timer expires, timer stopped
 - ◆ If timer goes off, segment retransmitted
- ◆ Sequence number
 - ◆ Used by receiver to identify *correct order of stream*

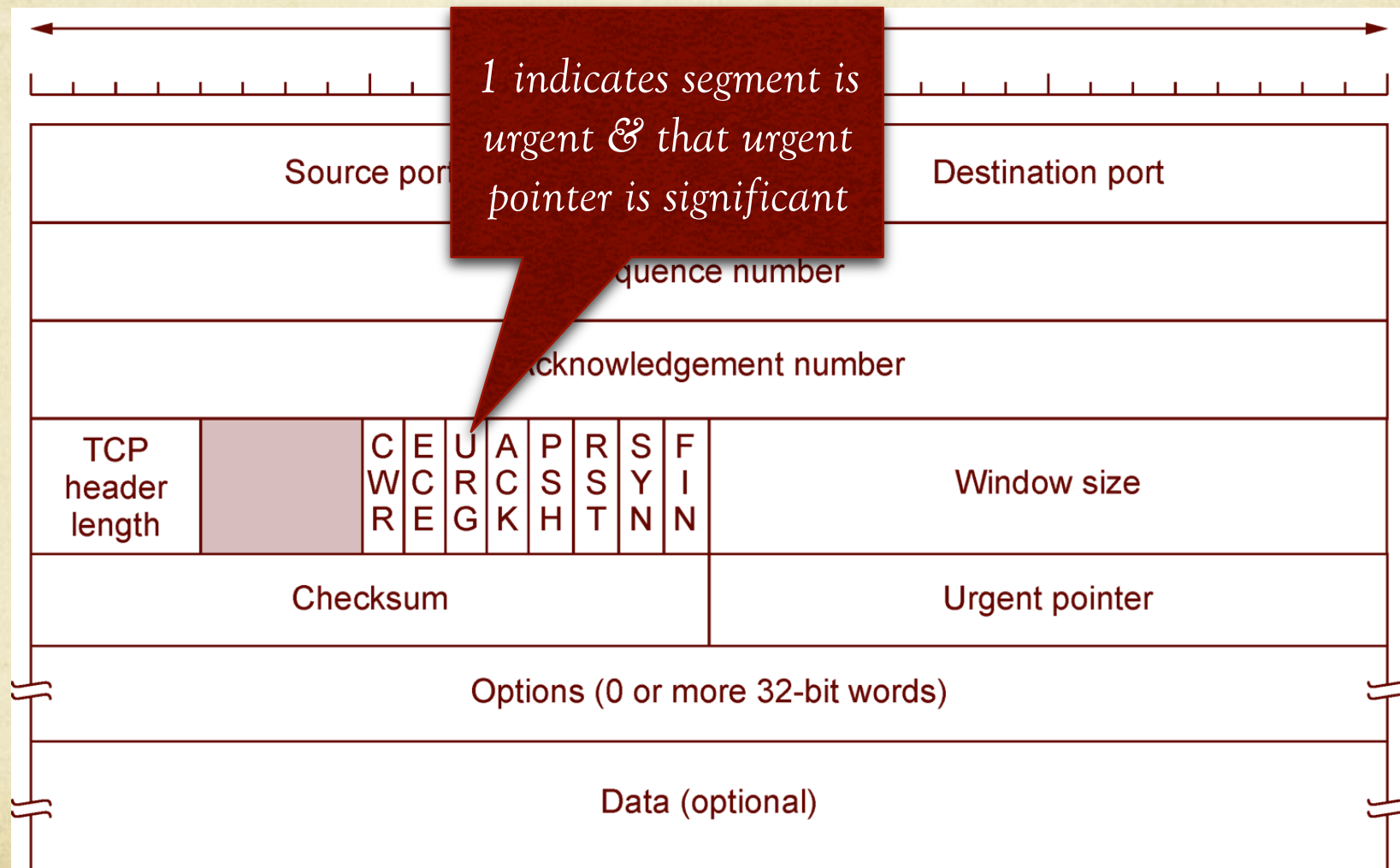
TCP header



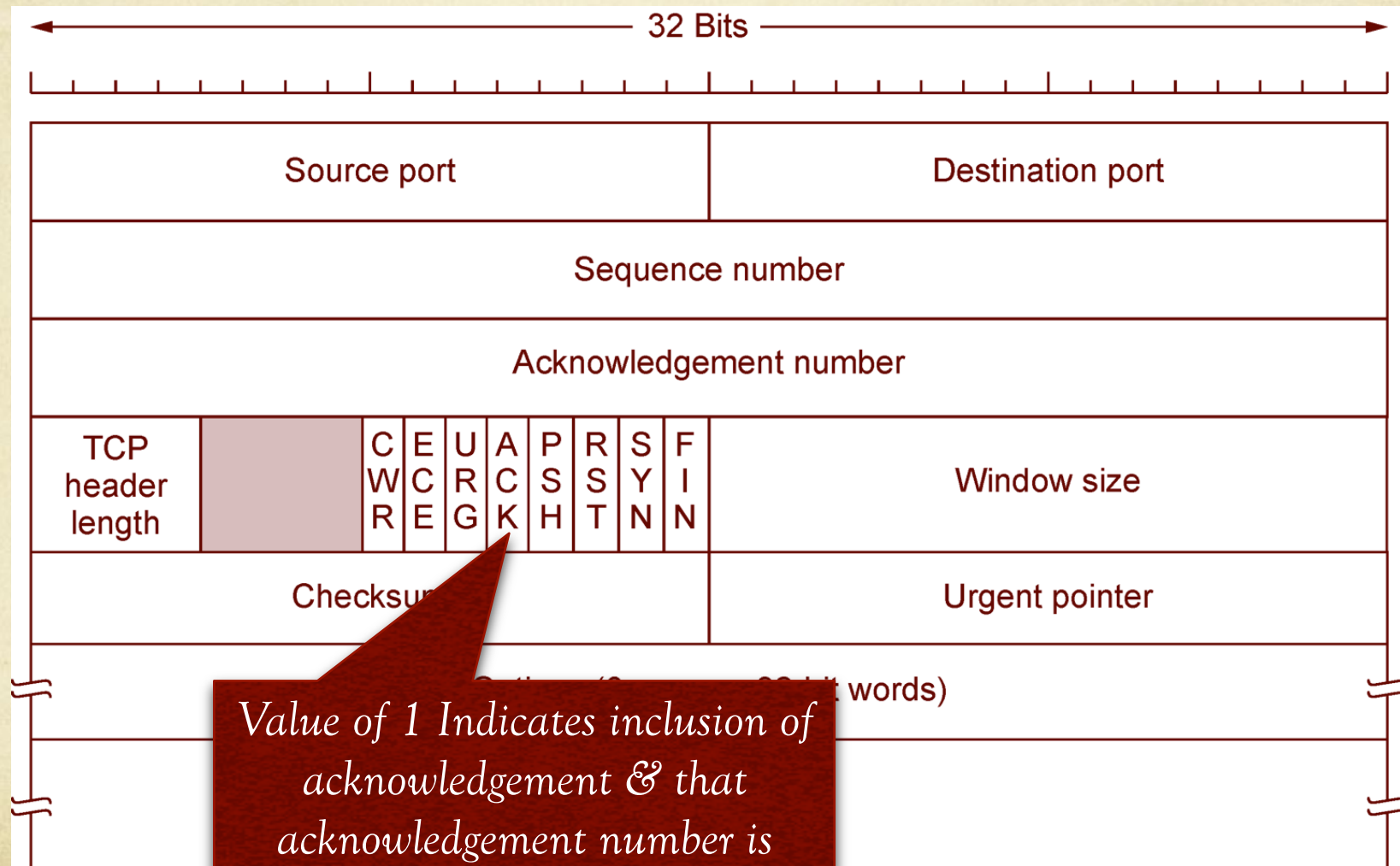
TCP header



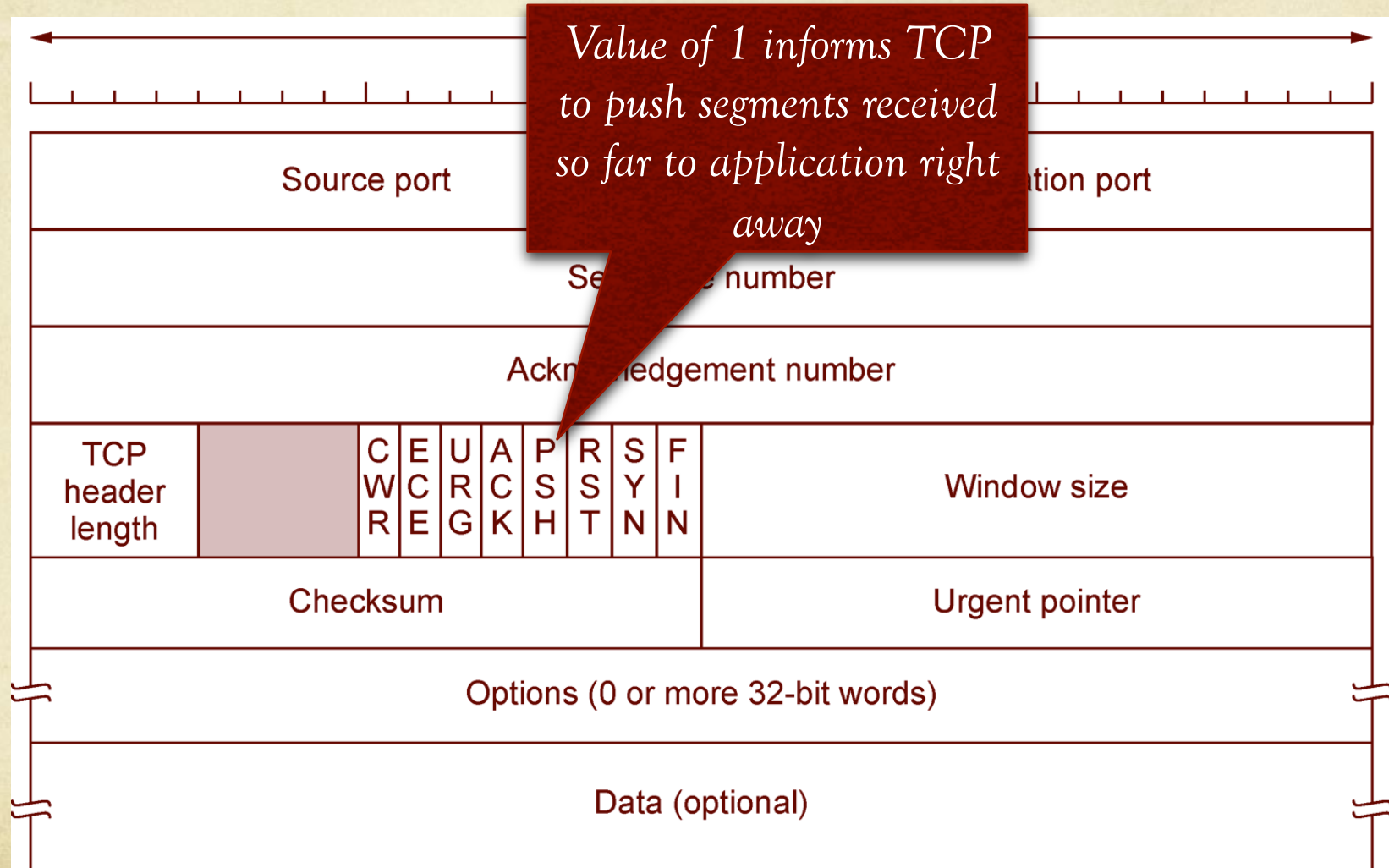
TCP header



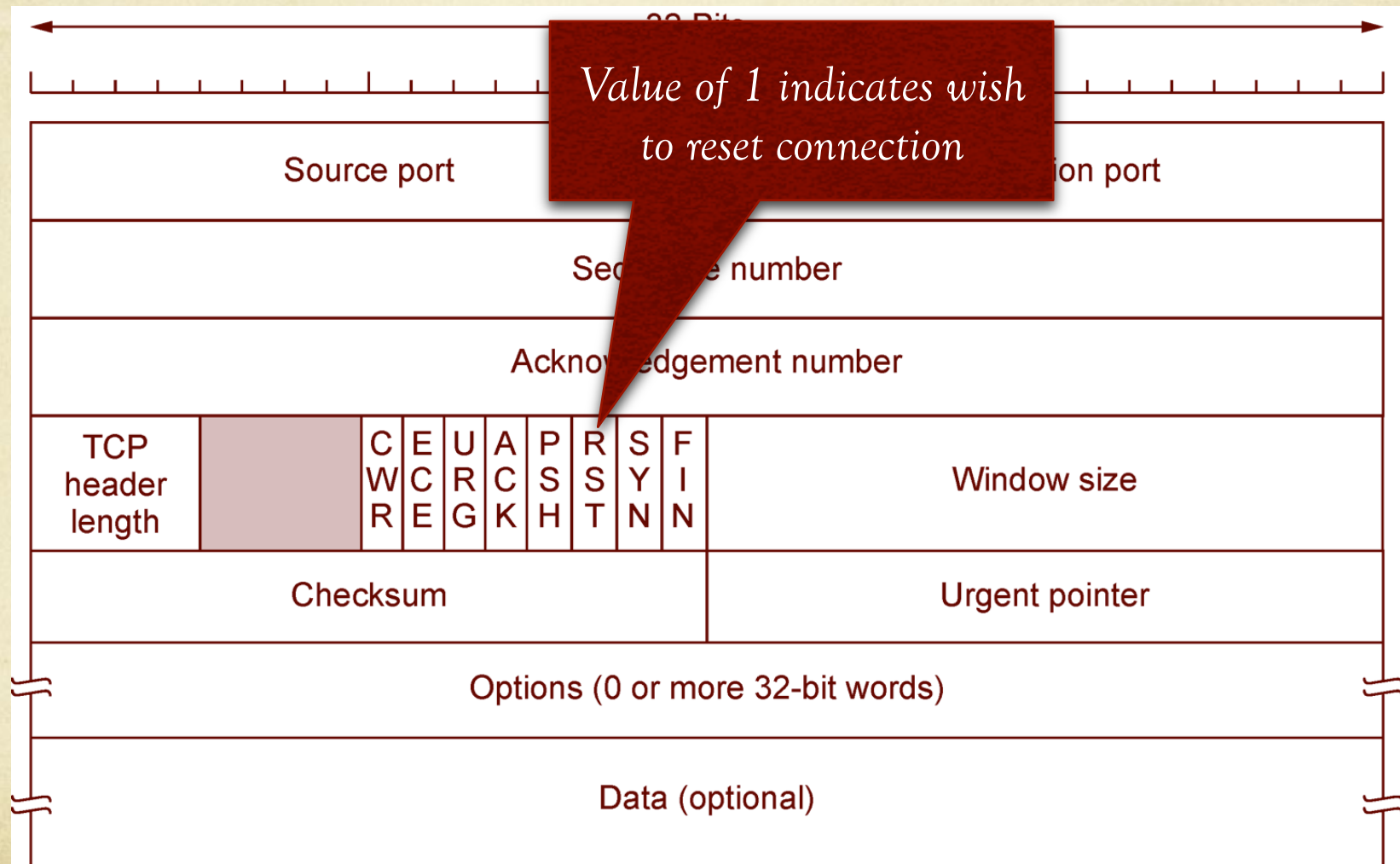
TCP header



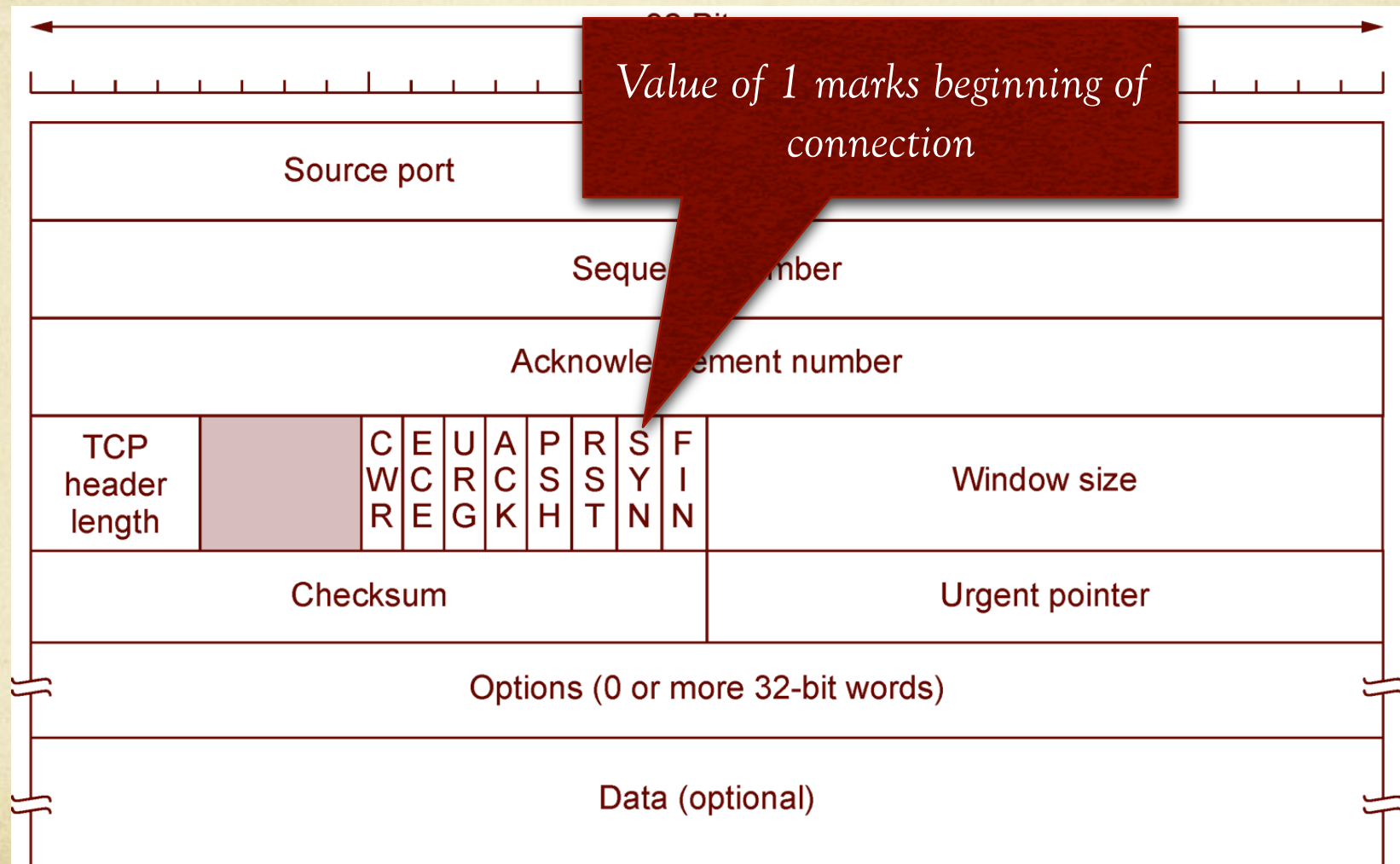
TCP header



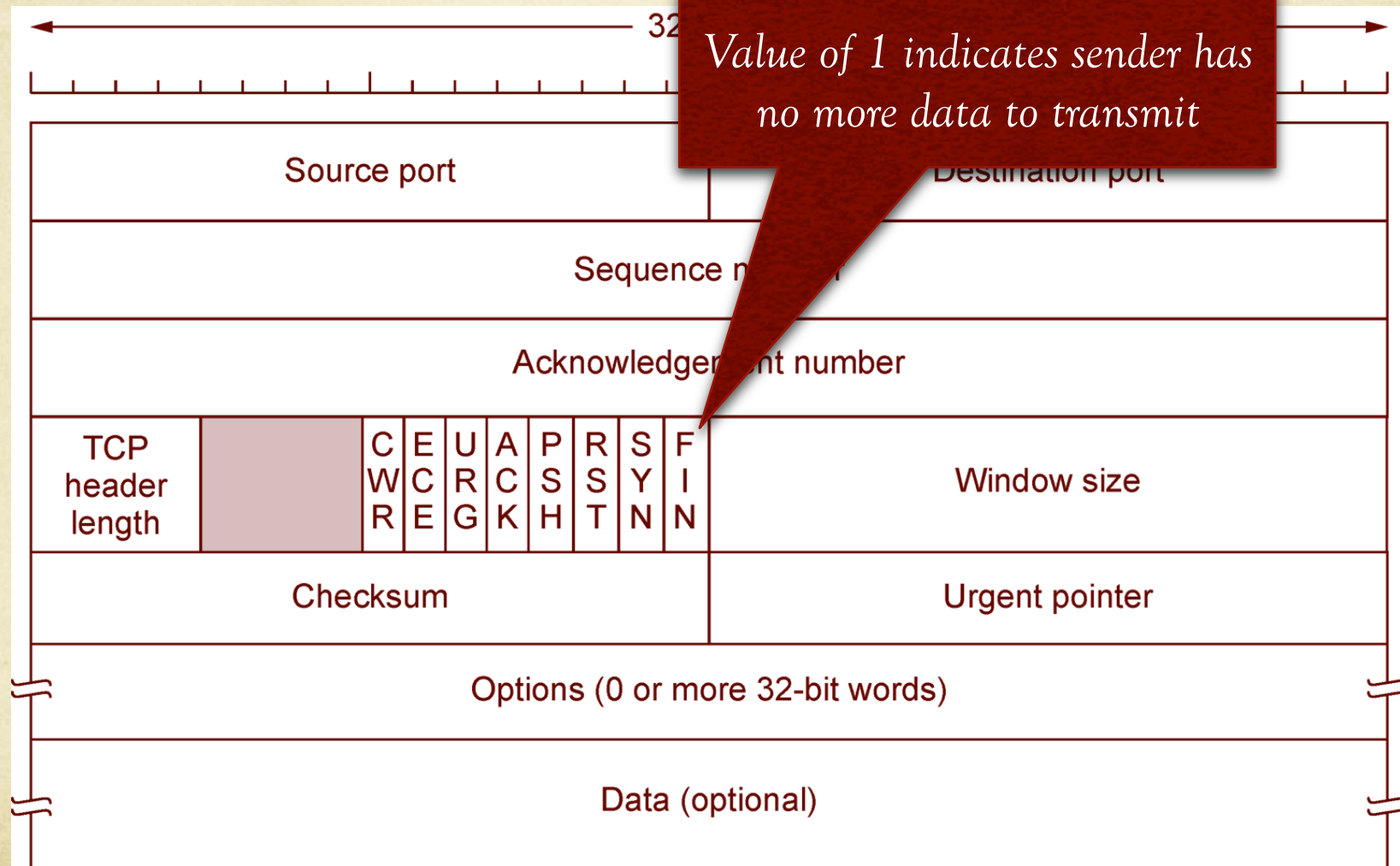
TCP header



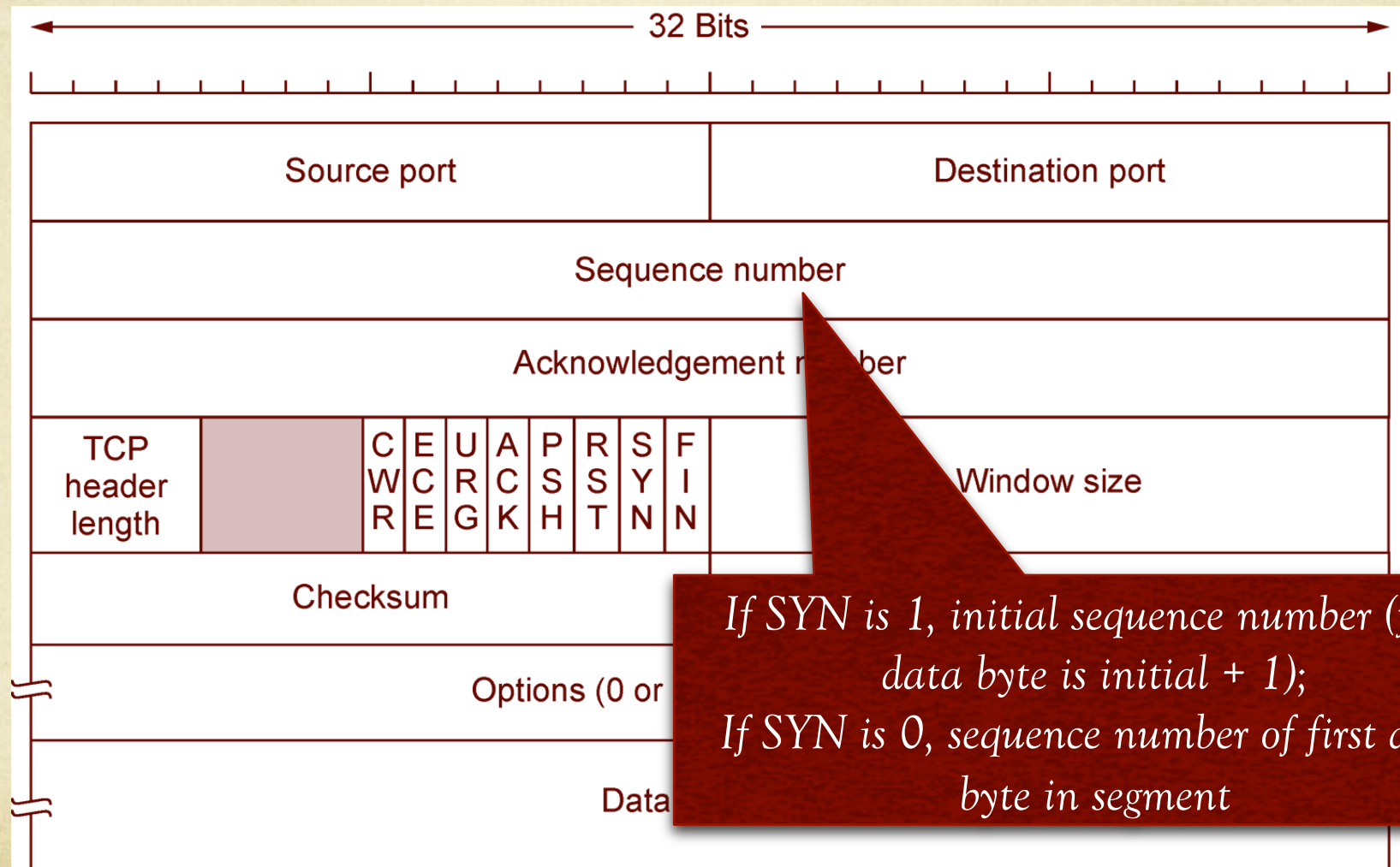
TCP header



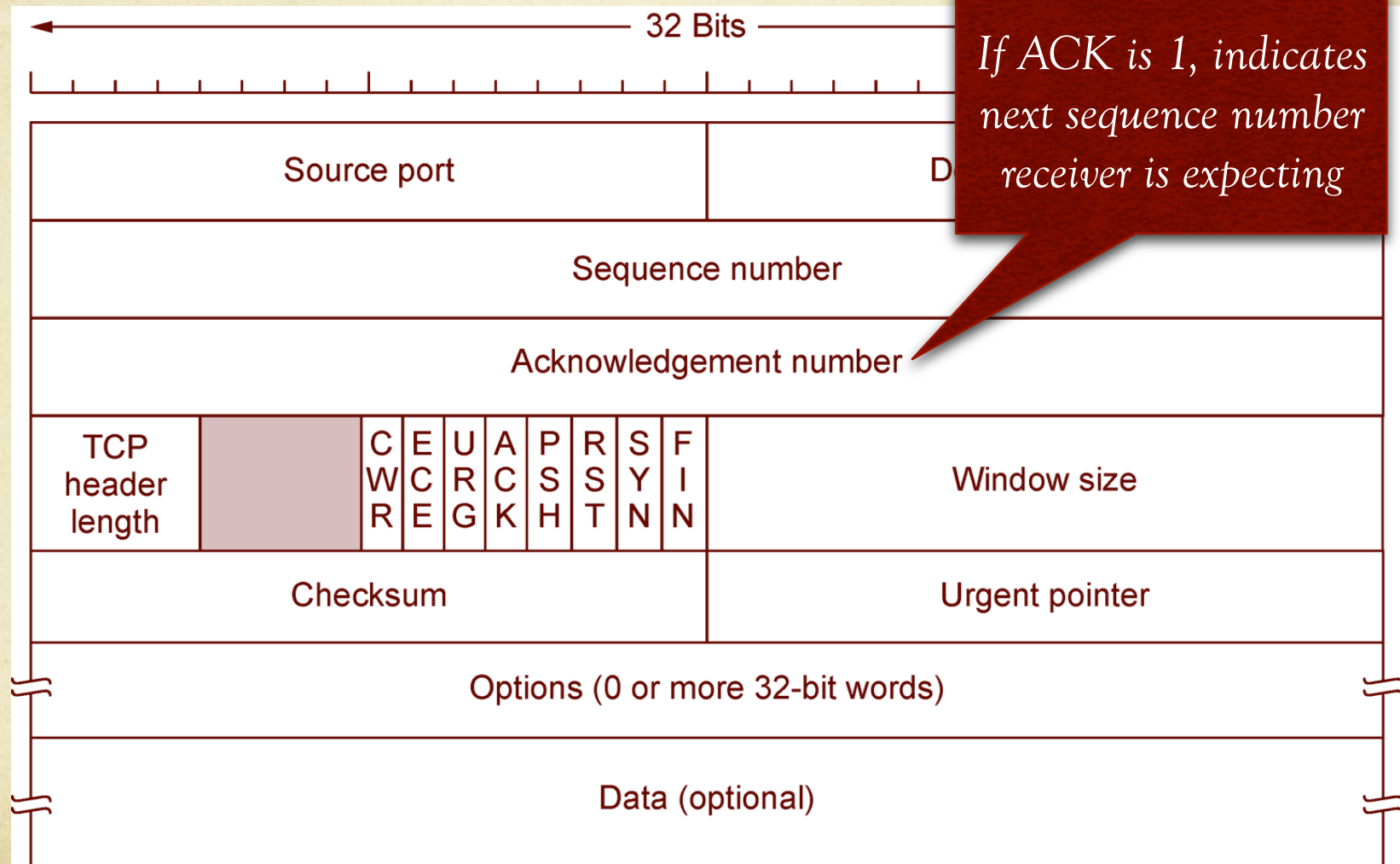
TCP header



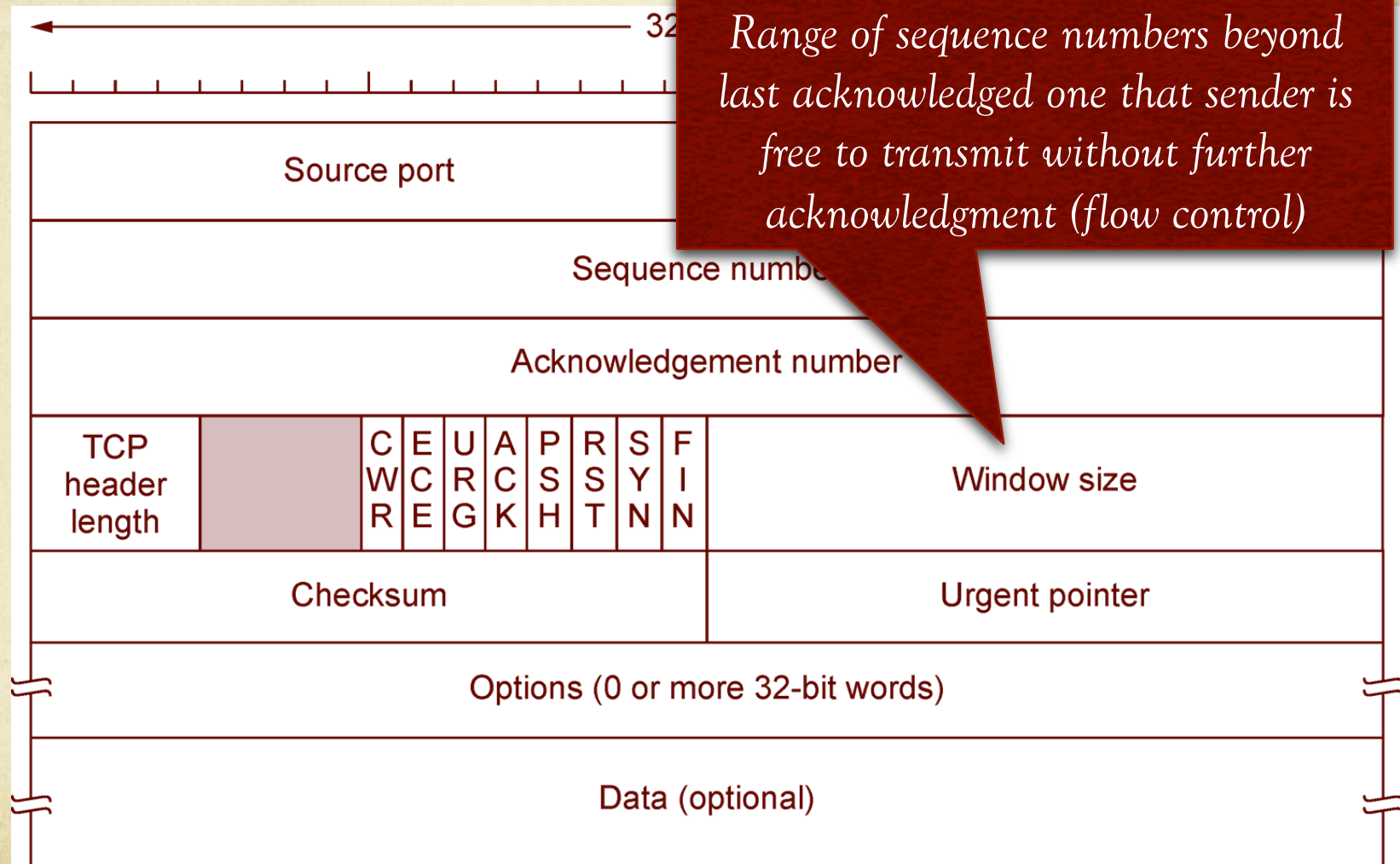
TCP header



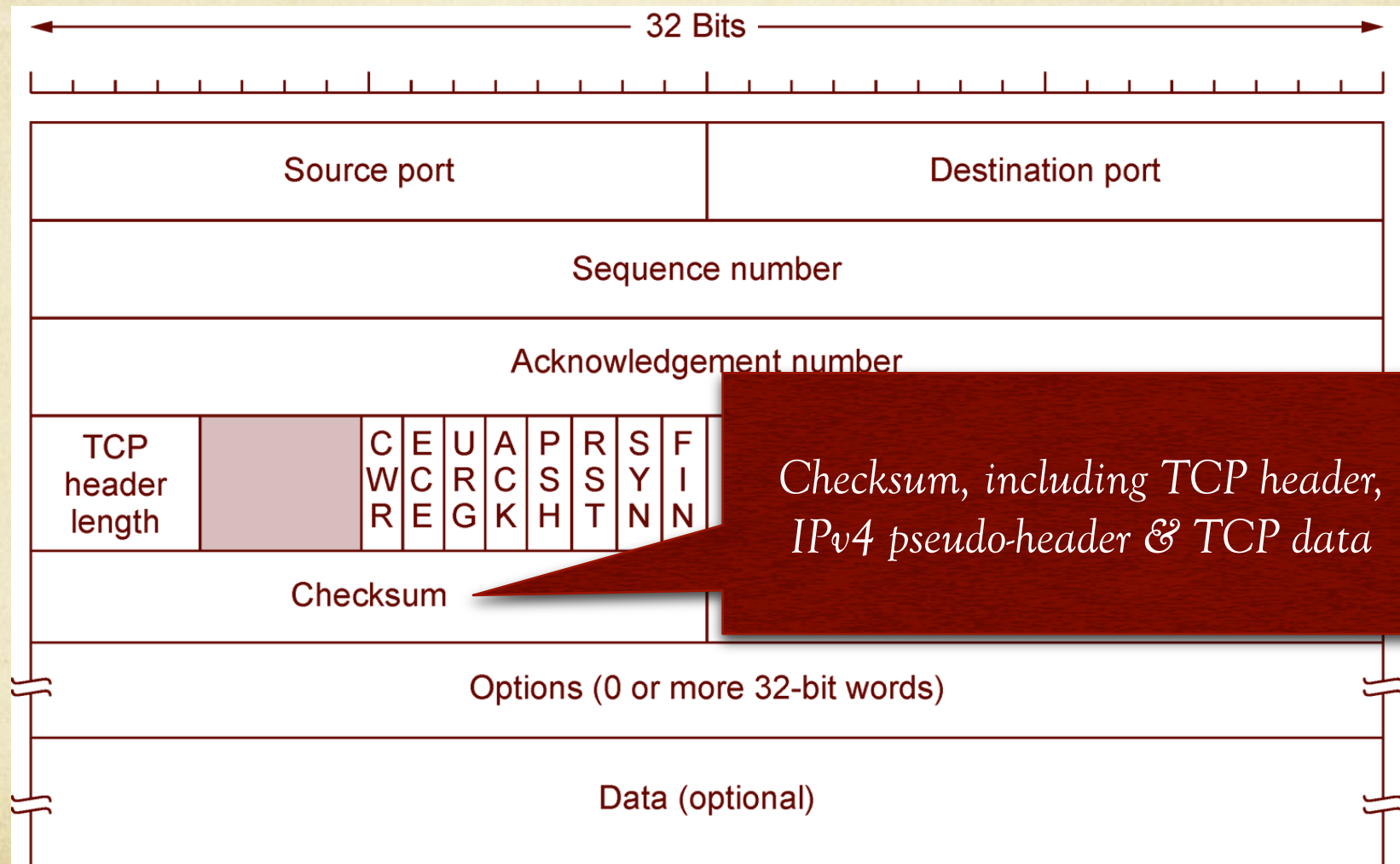
TCP header



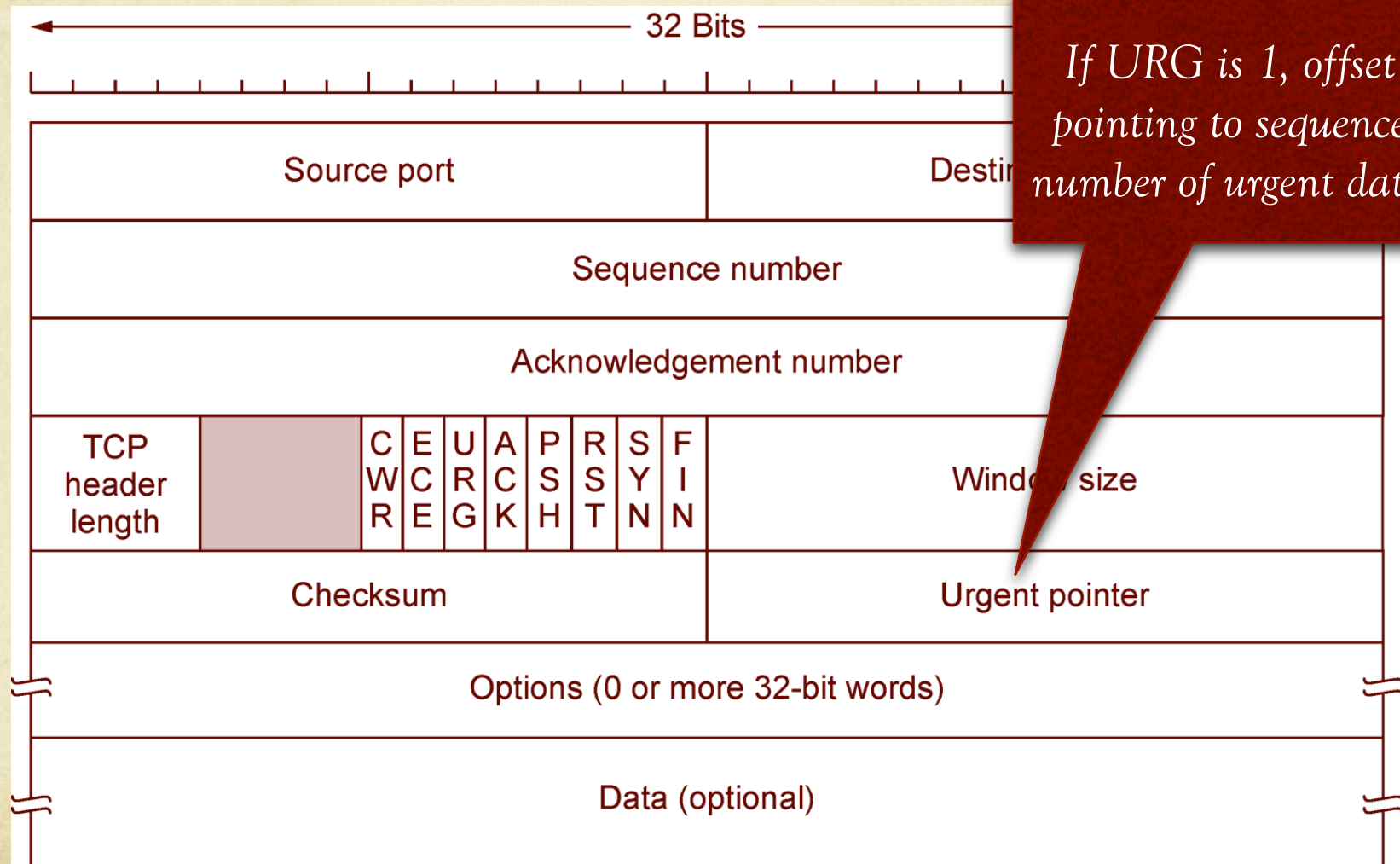
TCP header



TCP header



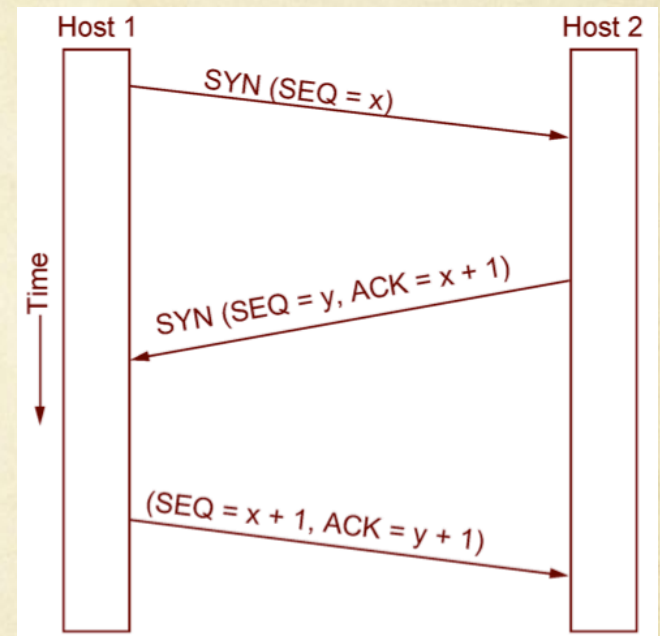
TCP header



If URG is 1, offset pointing to sequence number of urgent data

Connections in TCP

- ◆ *3-way handshake* to start connection
 - ◆ Host 2 → *passive* open state
 - ◆ Host 1 → requests new *connection*
 - ◆ *Host 1* sends segment with SYN = 1 & SEQ num = x
 - ◆ *Host 2* responds with ACK = 1, ACK num = x + 1, SYN = 1, & SEQ num = y
 - ◆ *Host 1* sends segment with SYN = 0, ACK flag = 1, ACK num = y + 1
 - ◆ Connection is now *active*!



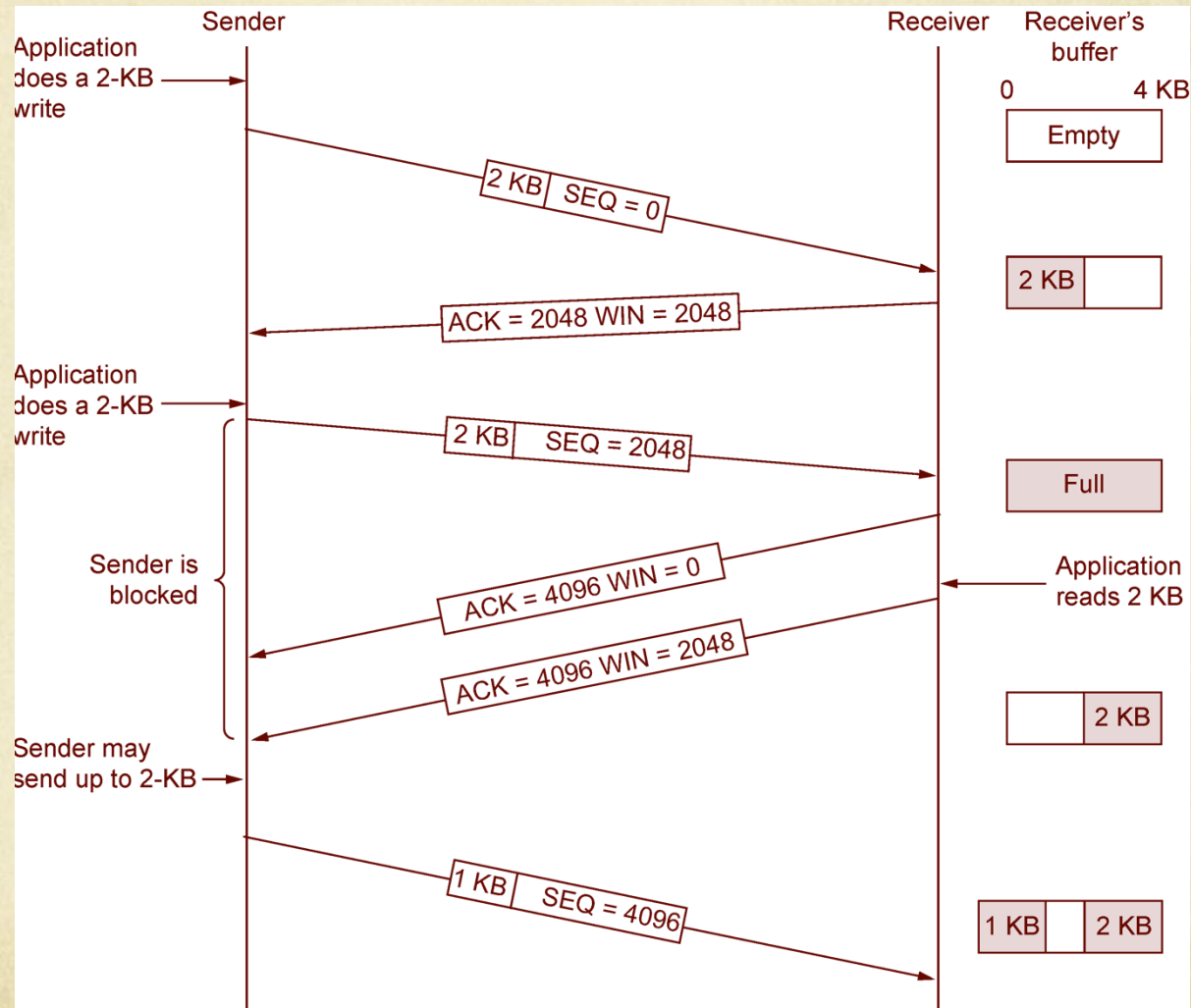
Connections in TCP

- ◆ Connection termination → *independently* for each direction
 - ◆ One computer (say Host 1) sends segment with *FIN* = 1
 - ◆ Other computer (say Host 2) *acknowledges* FIN
 - Communication stops in direction *Host 1* → *Host 2*
- ◆ *Pair* of *two-way* handshakes
- ◆ Each side uses *timer*
 - ◆ If *FIN* not acknowledged before timer expires, connection *released*

Flow control in TCP

- ◆ Flow control → ensure sender does not *inundate* receiver
- ◆ Mechanism used in TCP: *sliding window* protocol
 - ◆ *Window* field set by receiver
- ◆ Sequence numbers beyond *last acknowledged* one that sender is allowed to transmit before it receives more acknowledgements

Flow control in TCP



Congestion control in TCP

- ◆ Congestion control → avoid/recover from congestion
 - ◆ Network layer detects congestion & notifies sender
 - ◆ Transport layer on sender side adjusts transmission rate
 - Uses *congestion window* to restrict number of segments in network